

CHAPTER 1

1. Introduction

C++ is a general-purpose, Object Oriented Programming language developed by **Bjarne Stroustrup** at Bell Labs in 1979. C++ adds many new features to the C language, and that why considered as superset of C. It is similar to C language, and it is so compatible with C that it can run 99% of C programs without changing any source of code though C++ is an object-oriented programming language, so it is safer and well-structured programming language than C.

C++ runs on a variety of platforms, such as Windows, Mac OS, and the various versions of UNIX.

C++ is a middle-level language, as it encapsulates both high and low level language features.

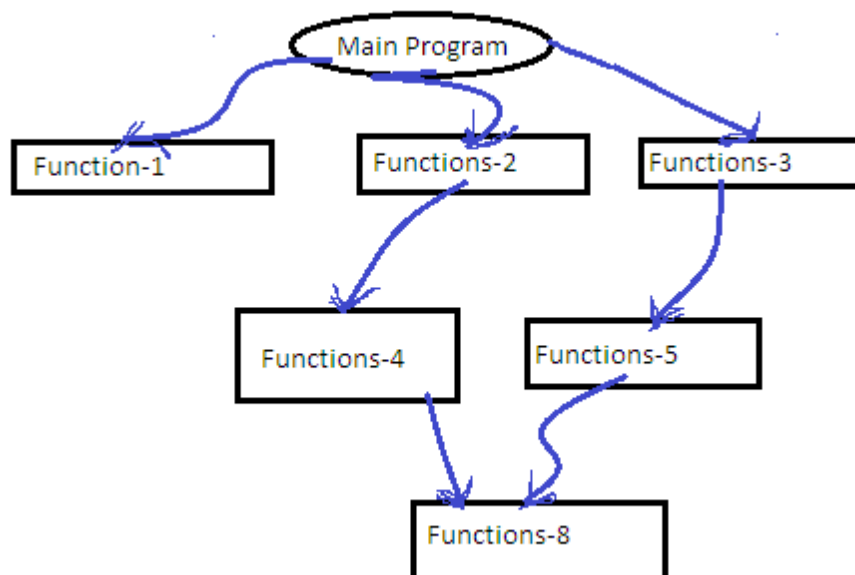
This language was updated 3 major times in 2011, 2014, and 2017 to C++11, C++14, and C++17.

It supports object-oriented, procedural and generic programming.

Generic programming is **a style of computer programming in which algorithms are written in terms of types to-be-specified-later** that are then instantiated when needed for specific types provided as parameters.

POP

Languages like Pascal, C, FORTRAN, and COBOL are called procedure oriented programming languages. Since in these languages, a programmer uses procedures or functions to perform a task. When the programmer wants to write a program, he will first divide the task into separate sub tasks, each of which is expressed as functions/ procedures. This approach is called procedure oriented approach.



A typical POP structure is shown in above:

In a multi-function program, many important data items are placed as global so that they may be accessed by all the functions. Each function may have its own local data. Global data are more vulnerable to an inadvertent change by a function. In a large program it is very difficult to identify what data is used by which function. In case we need to revise an external data structure, we should also revise all the functions that access the data. This provides an opportunity for bugs to creep in.

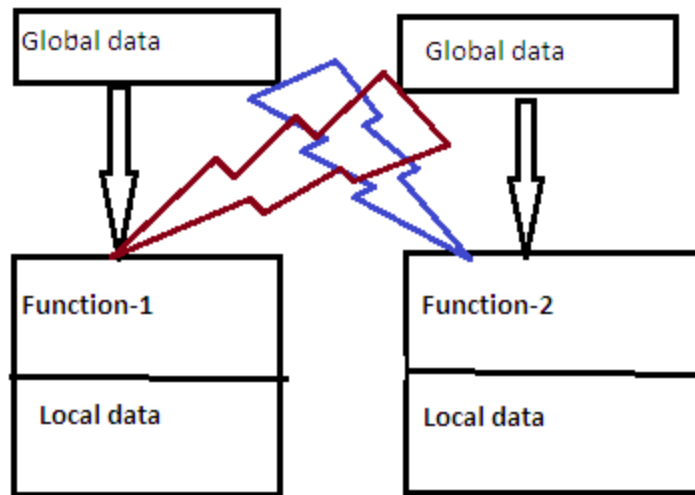
Drawback: It does not model real world problems very well, because functions are action oriented and do not really corresponding to the elements of the problem.

Characteristics of POP:

- Emphasis is on doing actions.
- Large programs are divided into smaller programs known as functions.
- Most of the functions shared global data.
- Data move openly around the program from function to function.
- Functions transform data from one form to another.
- Employs top-down approach in program design.

OOP:

The languages like C++ and Java use classes and object in their programs and are called Object Oriented Programming languages. The main task is divided into several modules and these are represented as classes. Each class can perform some tasks for which several methods are written in a class. This approach is called Object Oriented approach.

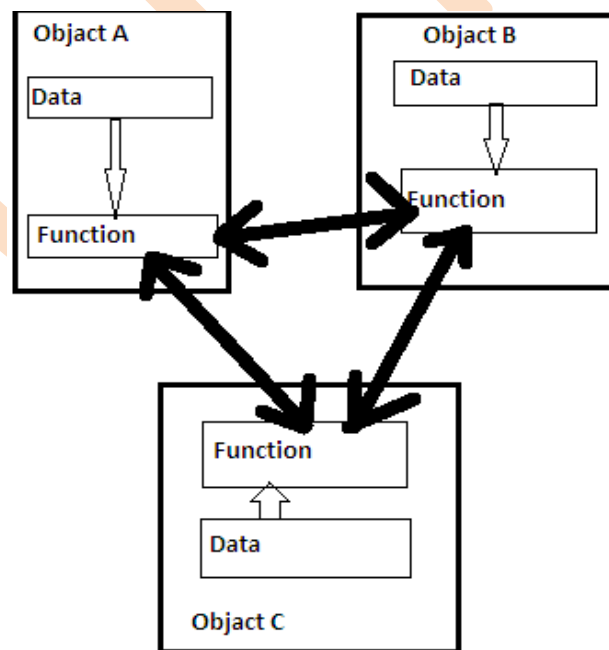


OOP allows us to decompose a problem into a number of entities called objects and then builds data and methods around these entities.

OOP is an approach that provides a way of modularizing programs by creating portioned memory area for both data and methods that can be used as templates for creating copies of such modules on demand.

That is, an object is considered to be a partitioned area of computer memory that stores data and set of operations that can access that data. Since the memory partitions are independent, the objects can be used in a variety of different programs without modifications.

Organization of OOP:



Characterstics OOP:

- Emphasis on data.
- Programs are divided into what are known as methods.
- Data structures are designed such that they characterize the objects.
- Methods that operate on the data of an object are tied together.
- Data is hidden.
- Objects can communicate with each other through methods.
- Reusability.
- Follows bottom-up approach in program design.

1.2 Why Learn C++

By the help of C++ programming language, we can develop different types of secured and robust applications:

- Window application
- Client-Server application
- Device drivers
- Embedded firmware etc

In computing, firmware is **a specific class of computer software that provides the low-level control for a device's specific hardware.** ... Some firmware memory devices are permanently installed and cannot be changed after manufacture. Common reasons for updating firmware include fixing bugs or adding features to the device.

1.2 C++ history

History of C++ language is interesting to know. Here we are going to discuss brief history of C++ language.

C++ programming language was developed in 1980 by Bjarne Stroustrup at bell laboratories of AT&T (American Telephone & Telegraph), located in U.S.A.

Bjarne Stroustrup is known as the **founder of C++ language.**



It was developed for adding a feature of **OOP (Object Oriented Programming)** in C without significantly changing the C component.

C++ programming is "relative" (called a superset) of C, it means any valid C program is also a valid C++ program.

1.3 C++ Features

C++ is object oriented programming language. It provides a lot of **features** that are given below.

- **Simple:** It is a simple language in the sense that programs can be broken down into logical units and parts, has a rich library support and a variety of data-types.
- **Machine Independent but Platform Dependent:** A C++ executable is not platform-independent (compiled programs on Linux won't run on Windows), however they are machine independent.
- **Mid-level language:** It is a mid-level language as we can do both systems-programming (drivers, kernels, networking etc.) and build large-scale user applications (Media Players, Photoshop, Game Engines etc.)
- **Rich library support:** Has a rich library support (Both standard ~ built-in data structures, algorithms etc.) as well 3rd party libraries (e.g. Boost libraries) for fast and rapid development.
- **Speed of execution:** C++ programs excel in execution speed. Since, it is a compiled language, and also hugely procedural. Newer languages have extra in-built default features such as garbage-collection, dynamic typing etc. which slow the execution of the program overall. Since there is no additional processing overhead like this in C++, it is blazing fast.
- **Pointer and direct Memory-Access:** C++ provides pointer support which aids users to directly manipulate storage address. This helps in doing low-level programming (where one might need to have explicit control on the storage of variables).
- **Object-Oriented:** One of the strongest points of the language which sets it apart from C. Object-Oriented support helps C++ to make maintainable and extensible programs. i.e. Large-scale applications can be built. Procedural code becomes difficult to maintain as code-size grows.
- **Compiled Language:** C++ is a compiled language, contributing to its speed.

1.4 C++ Compiler

- i. Bloodshed Dev-C++
- ii. Clang C++
- iii. Cygwin (GNU C++)
- iv. MINGW - "Minimalist GNU for Windows"
- v. GNU CC source
- vi. IBM C++
- vii. Intel C++
- viii. Microsoft Visual C++
- ix. Oracle C++
- x. HP C++

1.5 C++ IDE & Source Editor

- i. Visual Studio Code
- ii. Code:Block
- iii. CodeLite
- iv. Eclipse
- v. NetBeans
- vi. SubLime Text

1.6 C++ Standard Libraries

Standard C++ programming is divided into three important parts:

- The core library includes the data types, variables and literals, etc.
- The standard library includes the set of functions manipulating strings, files, etc.
- The Standard Template Library (STL) includes the set of methods manipulating a data structure.

1.7 C++ Program

Before starting the abcd of C++ language, you need to learn how to write, compile and run the first C++ program.

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World!";
```

```
    return 0;
}
```

Line 1: `#include <iostream>` is a **header file library** that lets us work with input and output objects, such as `cout` (used in line 5). Header files add functionality to C++ programs.

Line 2: using namespace `std` means that we can use names for objects and variables from the standard library.

Line 3: A blank line. C++ ignores white space.

Line 4: Another thing that always appear in a C++ program, is `int main()`. This is called a **function**. Any code inside its curly brackets `{ }` will be executed.

Line 5: `cout` (pronounced "see-out") is an **object** used together with the *insertion operator* (`<<`) to output/print text. In our example it will output "Hello World".

Note: Every C++ statement ends with a semicolon `;`.

Note: The body of `int main()` could also been written as:
`int main () { cout << "Hello World! "; return 0; }`

Remember: The compiler ignores white spaces. However, multiple lines makes the code more readable.

Line 6: `return 0` ends the main function.

Line 7: Do not forget to add the closing curly bracket `}` to actually end the main function.

1.8 Omitting Namespace

You might see some C++ programs that runs without the standard namespace library. The `using namespace std` line can be omitted and replaced with the `std` keyword, followed by the `::` operator for some objects:

```
#include <iostream>
int main() {
    std::cout << "Hello World!";
    return 0;
}
```

It is up to you if you want to include the standard namespace library or not.

1.9 C++ Basic Input/Output

C++ I/O operation is using the stream concept. Stream is the sequence of bytes or flow of data. It makes the performance fast.

If bytes flow from main memory to device like printer, display screen, or a network connection, etc, this is called as **output operation**.

If bytes flow from device like keyboard, files, or a network connection, etc to main memory, this is called as **input operation**.

Let us see the common header files used in C++ programming are:

Header File	Function and Description
<iostream>	It is used to define the cout , cin and cerr objects, which correspond to standard output stream, standard input stream and standard error stream, respectively.
<iomanip>	It is used to declare services useful for performing formatted I/O, such as setprecision and setw .
<fstream>	It is used to declare services for user-controlled file processing.

Standard output stream (cout)

The **cout** is a predefined object of **ostream** class. It is connected with the standard output device, which is usually a display screen. The cout is used in conjunction with stream insertion operator (<<) to display the output on a console

Let's see the simple example of standard output stream (cout):

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World!" <<endl<< "Good Morning;
    return 0;
}
```

You can add as many cout objects as you want. However, note that it does not insert a new line at the end of the output:

```
#include <iostream>
using namespace std;
```

```
int main() {  
    cout << "Hello World!";  
    cout << "I am learning C++";  
    return 0;  
}
```

C++ New Lines

To insert a new line, you can use the `\n` character:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    cout << "Hello World! \n";  
    cout << "I am learning C++";  
    return 0;  
}
```

Two <code>\n</code> characters after each other will create a blank line:
--

```
#include <iostream>  
using namespace std;  
  
int main() {  
    cout << "Hello World! \n\n";  
    cout << "I am learning C++";  
    return 0;  
}
```

Another way to insert a new line, is with the `endl` manipulator:

The **`endl`** is a predefined object of **`ostream`** class. It is used to insert a new line characters and flushes the stream.

```
#include <iostream>  
using namespace std;  
  
int main() {  
    cout << "Hello World!" << endl;  
    cout << "I am learning C++";  
    return 0;  
}
```

Output:

```
Hello World!  
I am learning C++
```

Standard input stream (cin)

The **cin** is a predefined object of **istream** class. It is connected with the standard input device, which is usually a keyboard. The cin is used in conjunction with stream extraction operator (>>) to read the input from a console.

Let's see the simple example of standard input stream (cin):

```
#include <iostream>  
using namespace std;  
int main( ) {  
    int age;  
    cout << "Enter your age: ";  
    cin >> age;  
    cout << "Your age is: " << age << endl;  
}
```

It is also possible to use the extraction operator >> on cin to read a string entered by a user:

```
string firstName;  
cout << "Type your first name: ";  
cin >> firstName; // get user input from the keyboard  
cout << "Your name is: " << firstName;
```

```
// Type your first name: John  
// Your name is: John
```

However, cin considers a space (whitespace, tabs, etc) as a terminating character, which means that it can only display a single word (even if you type many words):

```
string fullName;  
cout << "Type your full name: ";  
cin >> fullName;  
cout << "Your name is: " << fullName;
```

```
// Type your full name: John Doe  
// Your name is: John
```

From the example above, you would expect the program to print "John Doe", but it only prints "John".

That's why, when working with strings, we often use the **getline()** function to read a line of text. It takes cin as the first parameter, and the string variable as second:

```
string fullName;  
cout << "Type your full name: ";  
getline (cin, fullName);  
cout << "Your name is: " << fullName;
```

```
// Type your full name: John Doe  
// Your name is: John Doe
```

1.10 C++ Namespaces

Consider a situation where there are two individuals named Abhinav in the same class. To differentiate between them, we might use additional information such as their area of residence or their parents' names. This helps us identify which Abhinav we are referring to.

A similar situation can occur in C++ programming. Imagine you have written a function called `info()`, and you are using a library that also has a function named `info()`. The compiler cannot distinguish between the two functions, leading to a conflict.

Namespaces are designed to resolve this issue. They provide additional context to distinguish between functions, classes, variables, and other identifiers with the same name but located in different libraries or parts of the program. Essentially, a namespace defines a scope that allows for the organization and management of code to avoid naming collisions.

Why Namespaces are Required

Namespaces are essential to prevent name conflicts and collisions in C++ programs. Here are some key advantages:

- **Avoid Name Collisions:** Namespaces prevent naming conflicts by providing a way to differentiate between identifiers with the same name. For example, if you have a function `info()` and a library you are using also has a function `info()`, a namespace ensures the compiler can distinguish between them.
- **Organize Code:** Namespaces help in organizing code by grouping related functions, classes, and variables together. This makes the code more readable and maintainable.

- **Standard Library Example:** The C++ standard library (std) is a prime example of namespace usage. All classes, methods, and templates in the standard library are declared within the std namespace. When writing a C++ program, we typically include the directive **using namespace std;** to use the standard library components without needing to prefix them with **std::**.

Example of Namespace Usage

Below is the C++ program illustrating the use of namespace with the same name of function and variables:

```
#include <iostream>
using namespace std;
namespace LibraryA {
    void info() {
        cout << "Info from LibraryA" << endl;
    }
}

namespace LibraryB {
    void info() {
        cout << "Info from LibraryB" << endl;
    }
}

int main() {
    LibraryA::info(); // Calls info() from LibraryA
    LibraryB::info(); // Calls info() from LibraryB
    return 0;
}
```

In the example above, LibraryA and LibraryB are namespaces that each contain a function named info(). By specifying the namespace, we can call the appropriate info() function without any ambiguity.

Namespaces are used to avoid name collisions in larger projects where multiple libraries might be included, each containing functions, classes, or variables with the same names. By defining these elements within a namespace, you provide a context that uniquely identifies them.

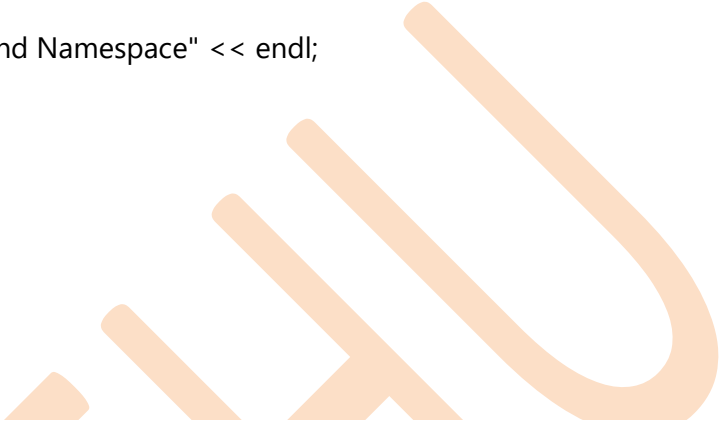
The using directive:

- You can also avoid prepending of namespaces with the using namespace directive. This directive tells the compiler that the subsequent code is making use of names in the specified namespace.
- The namespace is thus implied for the following code:

```

#include <iostream>
using namespace std;
namespace First{
    void sayHello(){
        cout << "Hello First Namespace" << endl;
    }
}
namespace Second{
    void sayHello(){
        cout << "Hello Second Namespace" << endl;
    }
}
using namespace First;
int main () {
    sayHello();
    return 0;
}

```



Output:

```

Hello First Namespace

```

Extending namespaces (Using same name twice)

It is also possible to create two namespace blocks having the same name. The second namespace block is nothing but actually the continuation of the first namespace. In simpler words, we can say that both the namespaces are not different but actually the same, which are being defined in parts.

```

// C++ program to demonstrate namespace extension
#include <iostream>
using namespace std;

// first name space
namespace first
{
    int val1 = 500;
}

// rest part of the first namespace

```

```

namespace first
{
int val2 = 501;
}

int main()
{
cout << first::val1 << "\n";
cout << first::val2 << "\n";
return 0;
}

```

Nested Namespaces

Namespaces can be nested within each other to create a hierarchy of namespaces. This is useful for organizing code into a more structured format.

```

#include <iostream>
namespace OuterNamespace {
    namespace InnerNamespace {
        void display() {
            std::cout << "Inside InnerNamespace" << std::endl;
        }
    }
}

int main() {
    OuterNamespace::InnerNamespace::display(); // Calls display from InnerNamespace
    return 0;
}

```

In this example, InnerNamespace is nested within OuterNamespace. We access the display function using OuterNamespace::InnerNamespace::display().

Another Example of Inner Namespace

```

#include <iostream>
using namespace std;

// first name space
namespace first_space
{
    void func()
    {

```

```

        cout << "Inside first_space" << endl;
    }

// second name space
namespace second_space
{
    void func()
    {
        cout << "Inside second_space" << endl;
    }
}
using namespace first_space::second_space;
int main ()
{
    // This calls function from second name space.
    func();

    return 0;
}

```

Using namespace in header files

We can create namespace in one file and access contents using another program. This is done in the following manner.

- We need to create two files. One containing the namespace and all the data members and member functions we want to use later.
- And the other program can directly call the first program to use all the data members and member functions in it.

File 1 (file1.h)

```

// file1.h
namespace foo
{
    int value()
    {
        return 5;
    }
}

```

File 2 (file2.cpp)

```

// file2.cpp - Not to be executed online
#include <iostream>
#include "file1.h" // Including file1
using namespace std;

```

```
int main ()
{
    cout << foo::value();
    return 0;
}
```

Here we can see that the namespace is created in file1.h and the value() of that namespace is getting called in file2.cpp.

The std Namespace

All the files in the C++ standard library declare all of its entities within the std namespace. However, it is possible that many of these entities (functions, objects, etc.) of the C++ standard might share identifiers with other namespaces in other libraries.

Or they might share identifiers with the ones we have created manually in our program.

This is why it is considered a bad practice to include the using namespace std; code in the global scope.

For example,

```
#include <iostream>
using namespace std;
```

```
string cout() {
    return "\n";
}
```

```
int main() {

    // Error: cout is ambiguous
    cout << "Testing!" << endl;

    return 0;
}
```

Here, we have defined our own cout() function while also using the std namespace. This causes a naming conflict in our program and we get an error when we use cout.

Why "using namespace" is Generally Considered Bad

- **Namespace Pollution:** It brings all identifiers from the namespace into the global scope, potentially causing name collisions.
- **Readability:** It can be unclear where a function or variable is coming from, making the code harder to read and maintain.

- **Maintainability:** If a new version of a library introduces a new identifier that clashes with an existing one, it can lead to bugs that are hard to trace.

Instead of using using namespace, it is better to use the namespace resolution operator :: to be explicit about which namespace an identifier belongs to:

```
#include <iostream>
```

```
int main() {  
    std::cout << "Hello, World!" << std::endl; // Explicit std:: prefix  
    return 0;  
}
```

This approach avoids the pitfalls of namespace pollution and makes the code more readable and maintainable. By explicitly specifying the namespace, you make it clear which library or context the identifiers are coming from.